

LeetCode 187: Repeated DNA Sequences

Authored by Arpit Raj, LNMIIT Jaipur • June 29, 2026

Problem Understanding

The DNA sequence is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and 'T'.

Given a string *s* that represents a DNA sequence, the task is to return all the **10-letter-long sequences** (substrings) that occur more than once in a DNA molecule. Multiple identical sequences might exist, but each unique repeating 10-letter sequence should be included in the output exactly once. The output is an array of strings.

Approach 1: Brute Force

Time Complexity: $O(N^2)$ | **Space Complexity:** $O(N)$

This approach involves iterating through every possible 10-letter window in the string and checking it against all subsequent 10-letter windows to find matches. We use an `unordered_set` to store our results so that we only store the unique occurrences of a common DNA sequence and avoid duplicates.

```
for (int i = 0; i <= n - 10; i++) {
    string s1 = s.substr(i, 10);
    for (int j = i + 1; j <= n - 10; j++) {
        string s2 = s.substr(j, 10);
        if (s1 == s2) {
            if (st.find(s1) == st.end()) {
                ans.push_back(s1);
                st.insert(s1);
            }
            break;
        }
    }
}
```

Approach 2: Sliding Window + 2 Hash Sets

Time Complexity: $O(N)$ | **Space Complexity:** $O(N)$

Because the window size is fixed at exactly 10, we can traverse all possible continuous windows of length 10. We check whether a specific window sequence has been seen before by maintaining two hash sets:

- **seen:** Tracks all unique sequences encountered as we slide our window.
- **added:** Ensures that a sequence already identified as a duplicate is not added to the answer array multiple times.

```
class Solution {
public:
    vector<string> findRepeatedDnaSequences(string s) {
        int n = s.size();
        vector<string> ans;
        unordered_set<string> seen;
        unordered_set<string> added;

        for(int l = 0; l + 9 < n; l++)
        {
            // Current DNA sequence
            string window = s.substr(l, 10);

            // Already seen before
            if(seen.count(window))
            {
                // Add only once to answer
                if(!added.count(window))
                {
                    ans.push_back(window);
                    added.insert(window);
                }
            }
            else
            {
                seen.insert(window);
            }
        }

        return ans;
    }
};
```

Approach 3: Sliding Window + HashMap Frequencies (Optimal)

Time Complexity: $O(N)$ | Space Complexity: $O(N)$

This is a more concise version of the optimal approach. Instead of maintaining two sets, we maintain a single `unordered_map` that stores the frequency of each 10-letter sequence. Whenever the frequency of a string sequence becomes **exactly 2**, we push it into the answer array. This naturally handles any further duplicates.

```
class Solution {
public:
    vector<string> findRepeatedDnaSequences(string s) {
        int n=s.size();
        unordered_map<string,int> mp;
        vector<string> ans;
        for(int i=0;i<=n-10;i++){
            string aadz=s.substr(i,10);
            mp[aadz]++;
            if(mp[aadz]==2){
                ans.push_back(aadz);
            }
        }
        return ans;
    }
};
```